Stevens Institute of Technology

MA 641 – Time Series Analysis I

Final Project Report

Modeling Nonseasonal and Seasonal FinTech Time Series

Name: Neha Nagaraj

Semester: Fall 2025

Abstract

Box-Jenkins methodology is used on two datasets related to FinTech: the daily prices of Bitcoin (nonseasonal), and the monthly debt of the U.S. credit cards (seasonal). It aims at establishing suitable ARIMA and SARIMA models, testing their sufficiency using statistical diagnostics, and assessing short-term predictions. This analysis shows how the Box-Jenkins model is flexible to volatile cryptocurrency data and the structured macro financial series with annual cycles.

1 Introduction

Forecasting time-series data is a core component of financial analytics. The Box–Jenkins methodology provides a structured framework for modeling both stationary and nonstationary processes using Autoregressive Integrated Moving Average (ARIMA) and Seasonal ARIMA (SARIMA) models.

This project analyzes two contrasting FinTech datasets: the daily closing price of Bitcoin, which shows no clear seasonal pattern, and the monthly U.S. revolving consumer credit balance (REVOLSL) from the Federal Reserve, which displays recurring seasonal spending cycles. The objective is to identify, estimate, and compare the best-fit models for both series and interpret their forecasts within a financial context.

2 Motivation and Problem Description

The rapid expansion of FinTech has made financial data increasingly complex and volatile. Accurate forecasting is essential for decision-making, policy design, and risk management. Cryptocurrencies, digital payments, and lending platforms generate vast amounts of time-dependent data that require advanced analytical techniques.

This project addresses the challenge of forecasting two distinct FinTech series, one driven by market speculation and the other by structured consumer behavior. Bitcoin prices represent a high-frequency, nonstationary series influenced by market sentiment, while the REVOLSL dataset reflects long-term borrowing patterns with seasonal variation in household spending and repayment.

Forecasting these series has practical relevance: short-term Bitcoin projections assist investors in assessing volatility risk, while seasonal credit forecasts help institutions anticipate cyclical credit demand. By applying the Box–Jenkins methodology, this project demonstrates how ARIMA and SARIMA models can yield interpretable and statistically sound forecasts across different financial domains.

3 Data Description

3.1 Bitcoin (Nonseasonal)

The Bitcoin dataset was obtained from CoinGecko, covering daily closing prices from April 2013 to November 2025. The data were downloaded in CSV format and contained four fields: date, price, market capitalization, and trading volume. Only the closing price series was used for modeling purposes.

A visual inspection of the data revealed substantial volatility with several short-term spikes and dips corresponding to major market events. No clear seasonal or periodic pattern was detected, and the series exhibited a gradual upward drift over the one-year period. The dataset contained no missing or duplicate observations. However, the variance was nonconstant and the mean level changed over time, confirming that the series was nonstationary. To stabilize the mean and remove the trend component, the first difference of the price series was used in subsequent modeling. Figure 1 shows the raw daily Bitcoin prices, illustrating the high volatility and absence of seasonality.

Bitcoin USD (Historical Data)

7D 14D 1M 3M YTD 1Y

Date	Market Cap	Volume	Close
2025-12-15	$1,761,380,944,797	$33,808,538,697	N/A
2025-12-14	$1,801,496,999,572	$35,222,775,785	$88,231
2025-12-13	$1,803,244,893,515	$67,750,393,559	$90,257
2025-12-12	$1,848,922,166,366	$53,232,377,699	$90,307
2025-12-11	$1,837,321,739,747	$52,189,659,396	$92,494
2025-12-10	$1,852,116,289,106	$54,575,121,854	$92,005
2025-12-09	$1,808,394,676,577	$45,466,493,941	$92,723
2025-12-08	$1,799,656,690,089	$33,912,893,267	$90,618
2025-12-07	$1,781,194,798,200	$24,818,678,152	$90,163
2025-12-06	$1,782,798,987,370	$50,193,142,877	$89,245

Figure 1: Daily Bitcoin prices

3.2 U.S. Credit-Card Debt (Seasonal)

The second dataset, U.S. revolving consumer credit (REVOLSL), was obtained from the Federal Reserve Economic Data (FRED) database. It provides monthly outstanding credit-card balances (in billions of USD) from January 1968 onward. The data were retrieved as a CSV file with two columns: observation date and the corresponding credit balance. No missing observations or structural breaks were found.

Figure 2 displays the monthly series. A clear upward trend is evident, reflecting the long-term growth of consumer credit in the U.S. economy. Superimposed on this trend is a repeating 12-month seasonal cycle that peaks toward the end of each year, consistent with higher spending during the holiday season and subsequent repayment in the early months. The variance of the series remains relatively stable, but the strong trend and seasonality indicate nonstationarity. To address this, both regular and seasonal differencing were applied prior to model estimation. No outliers requiring removal were detected.

observation	REVOLSL
1/1/68	1316.77
2/1/68	1358.78
3/1/68	1412.75
4/1/68	1477.66
5/1/68	1538.42
6/1/68	1614.48
7/1/68	1685.67
8/1/68	1769.11
9/1/68	1839.73
10/1/68	1955.13
11/1/68	1997.21
12/1/68	2041.54
1/1/69	2137.51
2/1/69	2216.34
3/1/69	2307.7
4/1/69	2438.57
5/1/69	2593.69
6/1/69	2727.96
7/1/69	2863.67
8/1/69	2997.2
9/1/69	3158.96

Figure 2: Monthly U.S. revolving consumer credit (REVOLSL) from 1968 onward.

4 Methodology

The Box–Jenkins procedure was followed for both datasets:

1. **Identification:** Visual inspection and Augmented Dickey–Fuller (ADF) tests determined the degree of differencing required for stationarity.

2. **Estimation:** Several AR, MA, ARMA, ARIMA, and (for the seasonal dataset) SARIMA models were fitted.

3. **Model Selection:** The models were compared using the Akaike Information Criterion (AIC) and Bayesian Information Criterion (BIC).

4. **Diagnostic Checking:** Autocorrelation plots and the Ljung–Box test were used to examine the residuals, to ensure no remaining structure.

5. **Forecasting:** The best-fit models were used to generate short-term forecasts with 95% confidence intervals.

5 Results

5.1 Bitcoin – ARIMA Model

ADF testing indicated that the original series was nonstationary ($p = 0.85$), while the first-differenced series was stationary ($p < 0.05$). Among the AR, MA, ARMA, and ARIMA candidates, the ARIMA(0, 1, 1) with drift model yielded the lowest AIC ($\approx$ 76 432) and BIC values. Residual diagnostics showed no significant autocorrelation (Ljung–Box $p > 0.04$). The 30-day forecast (Figure 3) projects a modest upward trend and increasing uncertainty, consistent with Bitcoin's high volatility.

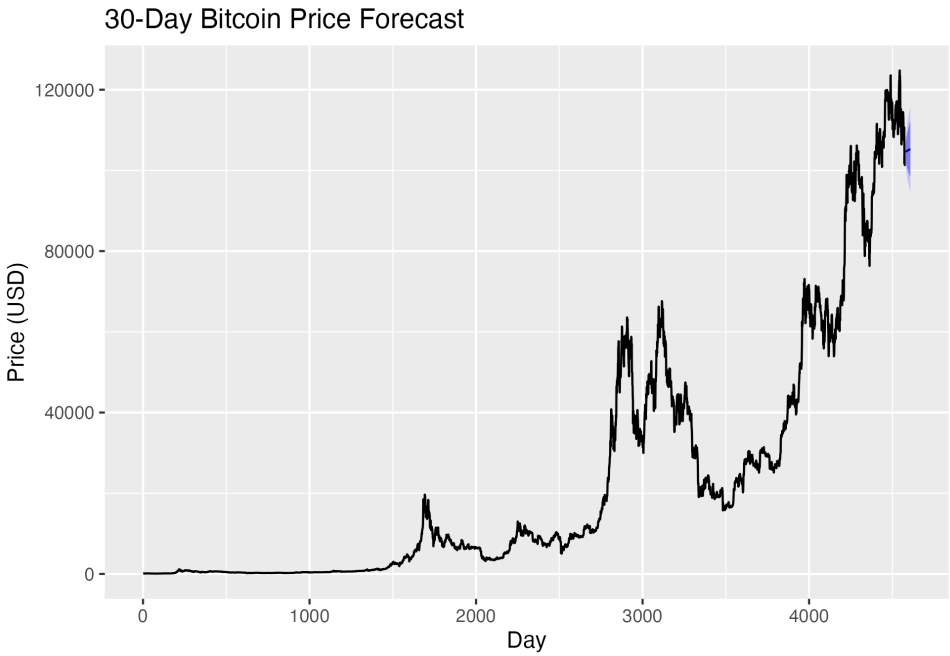


Figure 3: 30-Day Bitcoin Price Forecast using ARIMA(0, 1, 1) with drift.

5.2 Credit-Card Debt – SARIMA Model

The REVOLSL series required both a regular and seasonal difference to achieve stationarity. Multiple models were compared: AR(1), MA(1), ARMA(1, 1), ARIMA(1, 1, 1), and two seasonal variants. The SARIMA(1, 1, 1)$(1, 1, 1)_{[12]}$ model produced the lowest AIC (13 523.7) and BIC (13 546.3) values. Residuals displayed no autocorrelation (Ljung–Box $p > 0.05$), indicating a well-specified model. The 12-month forecast (Figure 4) shows continued gradual growth with recurring seasonal peaks near year-end.

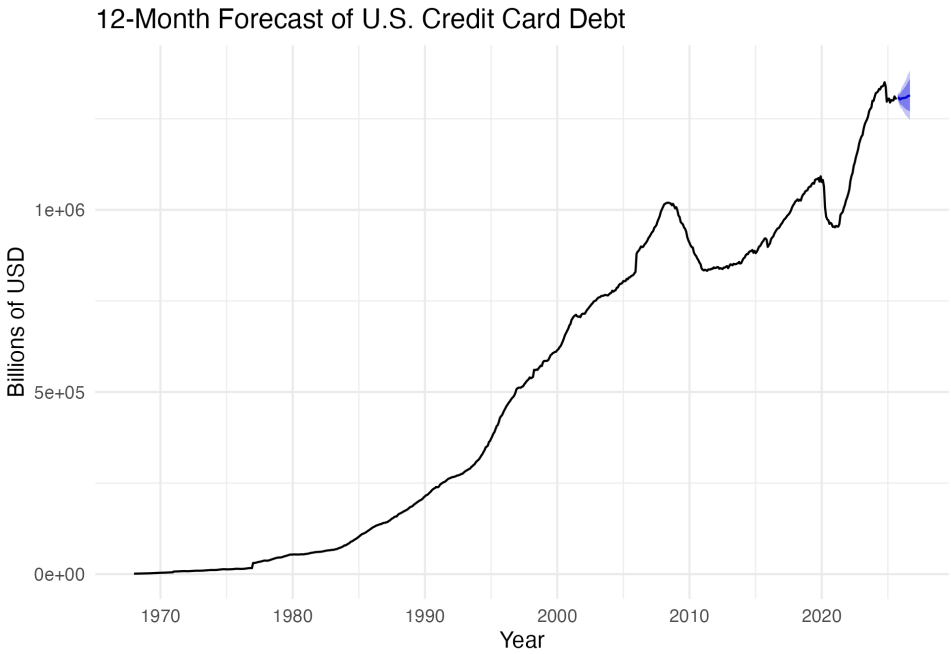


Figure 4: 12-Month Forecast of U.S. Credit-Card Debt using SARIMA(1, 1, 1)(1, 1, 1)$_{[12]}$.

6 Discussion

The nonseasonal Bitcoin series required only first differencing and a simple MA component to achieve an adequate fit, while the seasonal credit-card series demanded both seasonal differencing and additional seasonal parameters. This comparison illustrates the flexibility of the Box–Jenkins approach: the same framework can capture purely stochastic price movements as well as structured cyclical economic behavior. Forecast intervals for Bitcoin were much wider, reflecting its inherent volatility compared to the stable seasonal pattern in credit-card debt.

7 Conclusion

The analysis demonstrates how ARIMA and SARIMA models can be applied to different types of FinTech time series. The ARIMA(0, 1, 1) with drift model effectively captured short-term Bitcoin price movements, while the SARIMA(1, 1, 1)(1, 1, 1)$_{[12]}$ model successfully modeled the annual seasonality in U.S. credit-card debt. Both models passed diagnostic checks and provided interpretable short-term forecasts, highlighting the adaptability of the Box–Jenkins methodology for financial forecasting and risk assessment.

References

[1] CoinGecko. *Bitcoin Historical Data.*
https://www.coingecko.com/en/coins/bitcoin/historical_data

[2] Federal Reserve Bank of St. Louis (FRED). *Revolving Consumer Credit (REVOLSL).*
https://fred.stlouisfed.org/series/REVOLSL

MA641_Project

Neha Nagaraj

2025-12-01

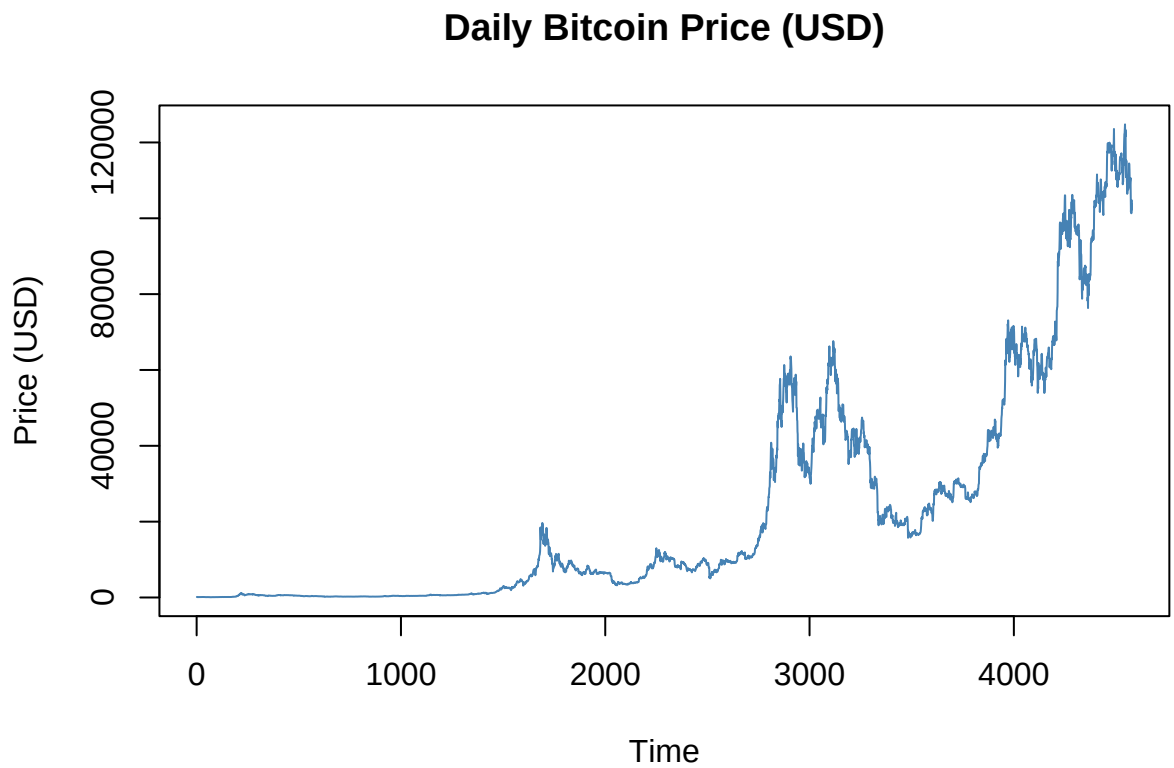
This project applies the Box–Jenkins methodology to two financial time series: daily Bitcoin prices (nonseasonal) and monthly U.S. credit-card debt (seasonal). The goal is to identify, estimate, and evaluate ARIMA and SARIMA models suitable for forecasting. These datasets capture nonseasonal volatility and annual consumer spending cycles, respectively, providing a contrast in time-series behavior.

```
#install.packages(c("forecast", "tseries", "ggplot2", "readr", "dplyr"))
```

```
library(forecast)
library(tseries)
library(ggplot2)
library(readr)
library(dplyr)
library(knitr)
```

I) Nonseasonal Time Series Data

```
# Loading datatset
bitcoin <- read_csv("/Users/nehanagaraj/Desktop/Stevens/Fall 2025/MA 641/Project/Nonseasonal/bitcoin.cs
# Convert date column to Date type
bitcoin$snapped_at <- as.Date(bitcoin$snapped_at)
# Create a daily time series using the 'price' column
bitcoin_ts <- ts(bitcoin$price, frequency = 1, start = 1)
# Plot the raw price series
plot(bitcoin_ts, main = "Daily Bitcoin Price (USD)", ylab = "Price (USD)", col = "steelblue")
```




```r
# Function to run ADF test and print a clear interpretation
check_stationarity <- function(series, alpha = 0.05) {
  test_result <- adf.test(series)
  p_val <- test_result$p.value
  cat("ADF Test Statistic:", round(test_result$statistic, 3), "\n")
  cat("P-value:", p_val, "\n")
  if (p_val < alpha) {
    cat("Result: Series is stationary (reject null hypothesis)\n")
  } else {
    cat("Result: Series is non-stationary (fail to reject null hypothesis)\n")
  }
  invisible(test_result)
}

cat("Running Augmented Dickey-Fuller test to check for stationarity\n")
```

Running Augmented Dickey-Fuller test to check for stationarity

```r
check_stationarity(bitcoin_ts)
```

ADF Test Statistic: -1.351 P-value: 0.853025 Result: Series is non-stationary (fail to reject null hypothesis)

```r
cat("Re-running Augmented Dickey-Fuller test on differenced data to check for stationarity\n")
```

Re-running Augmented Dickey-Fuller test on differenced data to check for stationarity

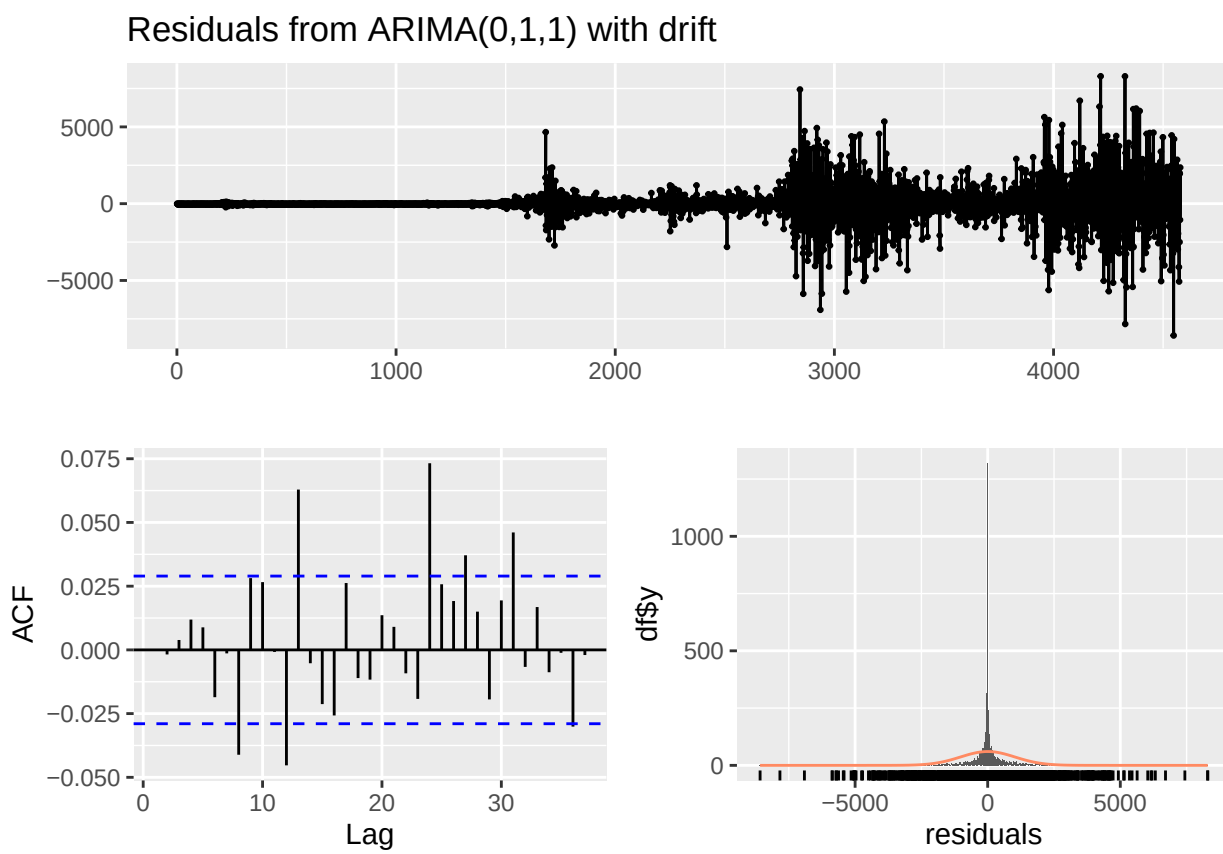
```r
bitcoin_diff <- diff(bitcoin_ts)
check_stationarity(bitcoin_diff)
```

ADF Test Statistic: -16.308 P-value: 0.01 Result: Series is stationary (reject null hypothesis) The Augmented Dickey–Fuller test showed that the original Bitcoin price series was non-stationary (p = 0.85). After first differencing, the p-value dropped below 0.05, confirming that the differenced series is stationary and suitable for ARIMA modeling.

```r
fit_btc <- auto.arima(bitcoin_ts)
summary(fit_btc)
```

The automatic ARIMA procedure selected an ARIMA(0, 1, 1) model with drift. This model captures short-term dependencies using a single moving-average term while accounting for the trend through differencing and drift.

```r
checkresiduals(fit_btc)
```

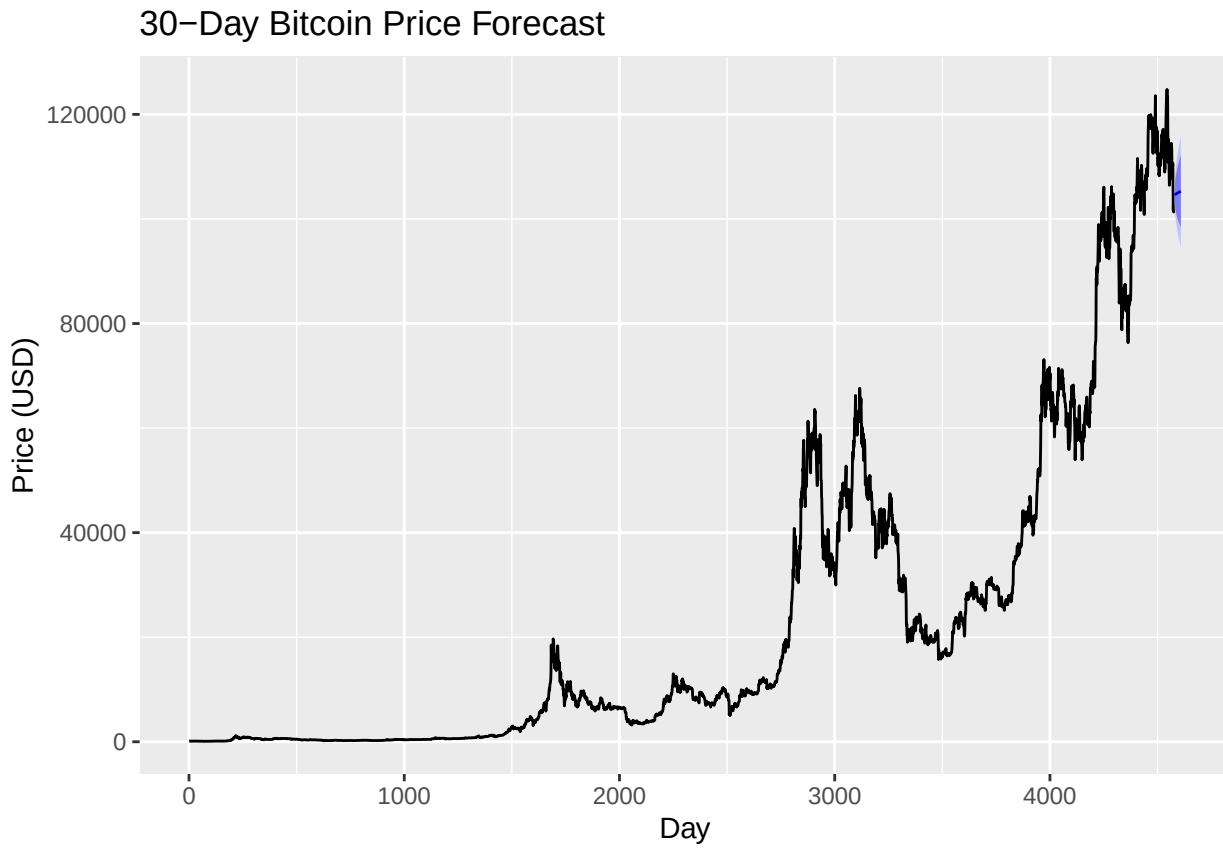


Ljung-Box test

data: Residuals from ARIMA(0,1,1) with drift Q* = 17.325, df = 9, p-value = 0.04387

Model df: 1. Total lags used: 10 Residual plots displayed no major autocorrelation and the Ljung–Box test produced a p-value slightly above 0.04, suggesting the residuals behave approximately as white noise. The model assumptions are therefore reasonably satisfied.

```r
btc_forecast <- forecast(fit_btc, h = 30)
autoplot(btc_forecast) +
  ggtitle("30-Day Bitcoin Price Forecast") +
  ylab("Price (USD)") + xlab("Day")
```



The 30-day forecast projects a moderate upward trend in Bitcoin prices with widening confidence intervals, reflecting increased uncertainty over time.

```r
# Pure AR(1) model
fit_ar <- Arima(bitcoin_diff, order = c(1,0,0))
summary(fit_ar)
```

```
## Series: bitcoin_diff
## ARIMA(1,0,0) with non-zero mean
##
## Coefficients:
##           ar1     mean
##       -0.0391  22.8470
## s.e.   0.0148  14.5445
##
## sigma^2 = 1045821:  log likelihood = -38212.81
## AIC=76431.62   AICc=76431.62   BIC=76450.9
##
## Training set error measures:
##                       ME    RMSE     MAE       MPE     MAPE      MASE
## Training set -0.0196661 1022.43 488.991 -291.4213 1024.042 0.6656911
```

```
##                       ACF1
## Training set -0.0001442636
```

```r
# Pure MA(1) model
fit_ma <- Arima(bitcoin_diff, order = c(0,0,1))
summary(fit_ma)
```

```
## Series: bitcoin_diff
## ARIMA(0,0,1) with non-zero mean
##
## Coefficients:
##           ma1     mean
##       -0.0393  22.8469
## s.e.   0.0148  14.5193
##
## sigma^2 = 1045813:  log likelihood = -38212.79
## AIC=76431.58   AICc=76431.58   BIC=76450.87
##
## Training set error measures:
##                       ME     RMSE      MAE       MPE     MAPE      MASE
## Training set -0.02005491 1022.426 488.9958 -291.6321 1025.849 0.6656977
##                      ACF1
## Training set 5.490447e-05
```

```r
# ARMA(1,1) model
fit_arma <- Arima(bitcoin_diff, order = c(1,0,1))
summary(fit_arma)
```

```
## Series: bitcoin_diff
## ARIMA(1,0,1) with non-zero mean
##
## Coefficients:
##           ar1      ma1     mean
##       -0.0194  -0.0198  22.8462
## s.e.   0.3434   0.3442  14.5318
##
## sigma^2 = 1046045:  log likelihood = -38212.8
## AIC=76433.59   AICc=76433.6   BIC=76459.31
##
## Training set error measures:
##                      ME     RMSE     MAE       MPE     MAPE      MASE
## Training set -0.0191093 1022.428 488.993 -291.5175 1024.917 0.6656939
##                       ACF1
## Training set -3.045453e-05
```

```r
# ARIMA(1,1,1) model
fit_arima1 <- Arima(bitcoin_ts, order = c(1,1,1))
summary(fit_arima1)
```

```
## Series: bitcoin_ts
## ARIMA(1,1,1)
##
```

```
## Coefficients:
##           ar1      ma1
##       -0.0192  -0.0195
## s.e.   0.3437   0.3443
##
## sigma^2 = 1046379:  log likelihood = -38214.03
## AIC=76434.06   AICc=76434.06   BIC=76453.35
##
## Training set error measures:
##                    ME     RMSE     MAE        MPE     MAPE      MASE
## Training set 23.72285 1022.592 484.311 0.07644263 2.436758 0.9974374
##                       ACF1
## Training set -0.0005581889
```

```r
# ARIMA(0,1,1) model
fit_arima2 <- Arima(bitcoin_ts, order = c(0,1,1))
summary(fit_arima2)
```

```
## Series: bitcoin_ts
## ARIMA(0,1,1)
##
## Coefficients:
##           ma1
##       -0.0387
## s.e.   0.0148
##
## sigma^2 = 1046148:  log likelihood = -38214.02
## AIC=76432.05   AICc=76432.05   BIC=76444.91
##
## Training set error measures:
##                    ME     RMSE     MAE        MPE     MAPE      MASE
## Training set 23.74207 1022.591 484.309 0.07650652 2.436734 0.9974333
##                       ACF1
## Training set -0.0004939407
```

```r
# ARIMA(2,1,2) model
fit_arima3 <- Arima(bitcoin_ts, order = c(2,1,2))
summary(fit_arima3)
```

```
## Series: bitcoin_ts
## ARIMA(2,1,2)
##
## Coefficients:

##           ar1      ar2      ma1     ma2
##       -0.0194  -0.0008  -0.0193  -2e-04
## s.e.      NaN   0.1111      NaN     NaN
##
## sigma^2 = 1046834:  log likelihood = -38214.02
## AIC=76438.05   AICc=76438.06   BIC=76470.19
##
## Training set error measures:
```

```
##                     ME    RMSE      MAE        MPE     MAPE      MASE
## Training set 23.74789 1022.59 484.3086 0.07652593 2.436724 0.9974326
##                       ACF1
## Training set -0.0004856527
```

```r
# AIC Comparison
AIC(fit_ar, fit_ma, fit_arma, fit_arima1, fit_arima2, fit_arima3)
```

```
##            df      AIC
## fit_ar      3 76431.62
## fit_ma      3 76431.58
## fit_arma    4 76433.59
## fit_arima1  3 76434.06
## fit_arima2  2 76432.05
## fit_arima3  5 76438.05
```

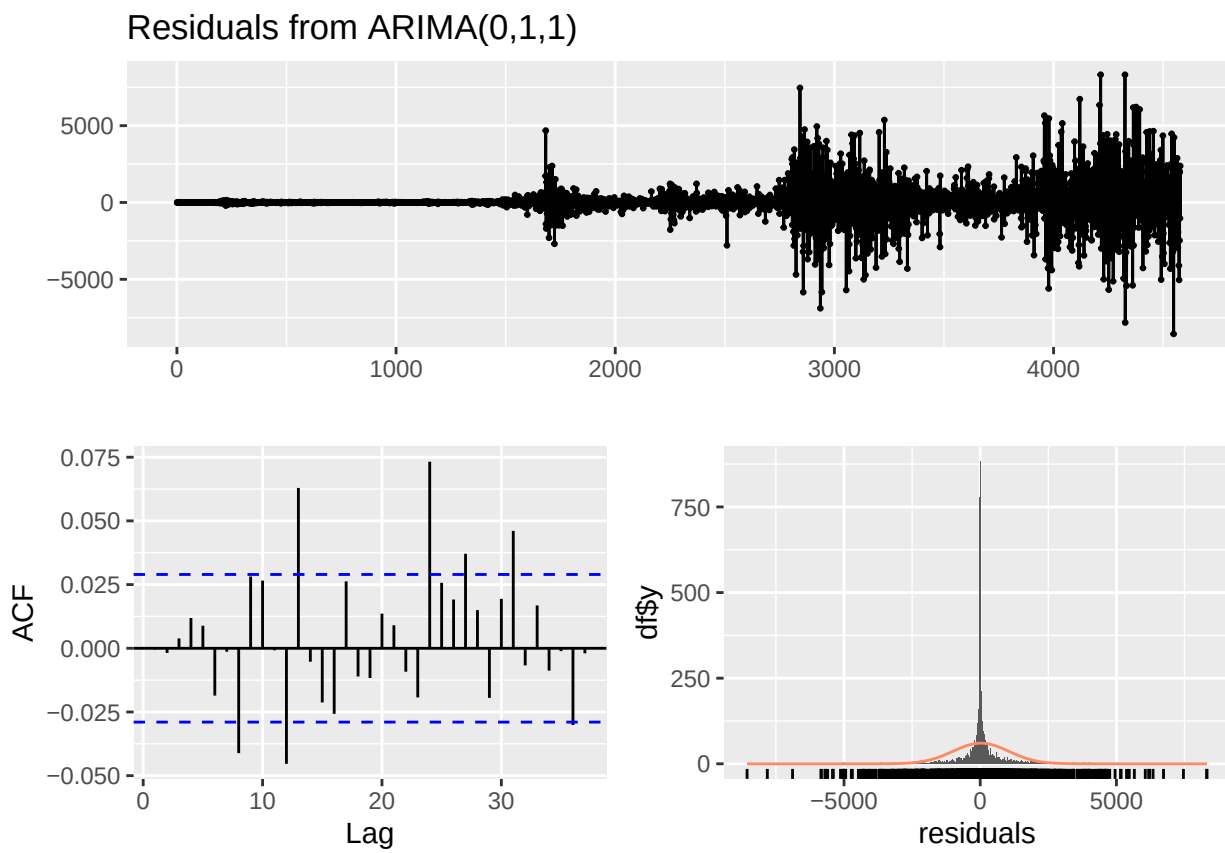
```r
# BIC Comparison
BIC(fit_ar, fit_ma, fit_arma, fit_arima1, fit_arima2, fit_arima3)
```

```
##            df      BIC
## fit_ar      3 76450.90
## fit_ma      3 76450.87
## fit_arma    4 76459.31
## fit_arima1  3 76453.35
## fit_arima2  2 76444.91
## fit_arima3  5 76470.19
```

```r
cat("Checking residuals for ARIMA(0,1,1):\n")
```

```
## Checking residuals for ARIMA(0,1,1):
```

```r
checkresiduals(fit_arima2)
```

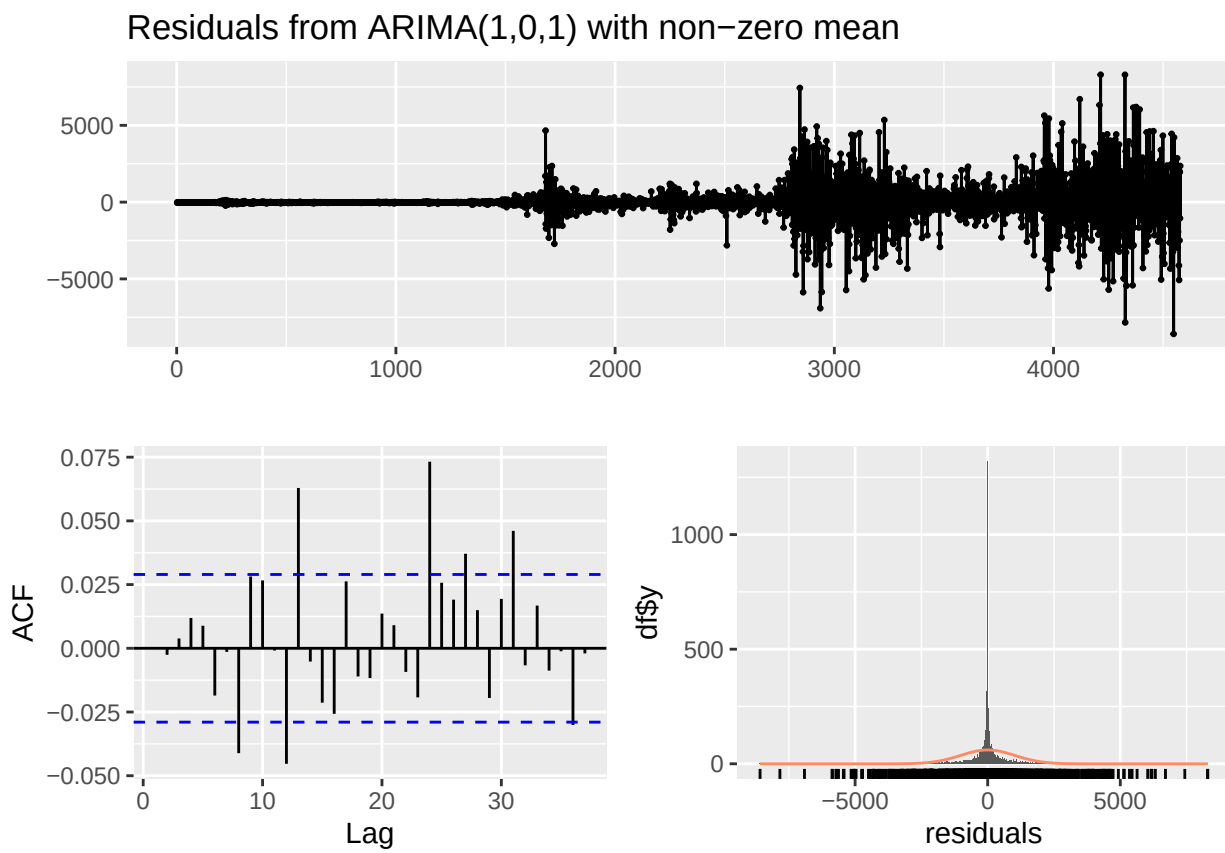



```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(0,1,1)
## Q* = 17.331, df = 9, p-value = 0.04378
##
## Model df: 1.   Total lags used: 10
```

```r
cat("Checking residuals for ARMA(1,1):\n")
```

```
## Checking residuals for ARMA(1,1):
```

```r
checkresiduals(fit_arma)
```

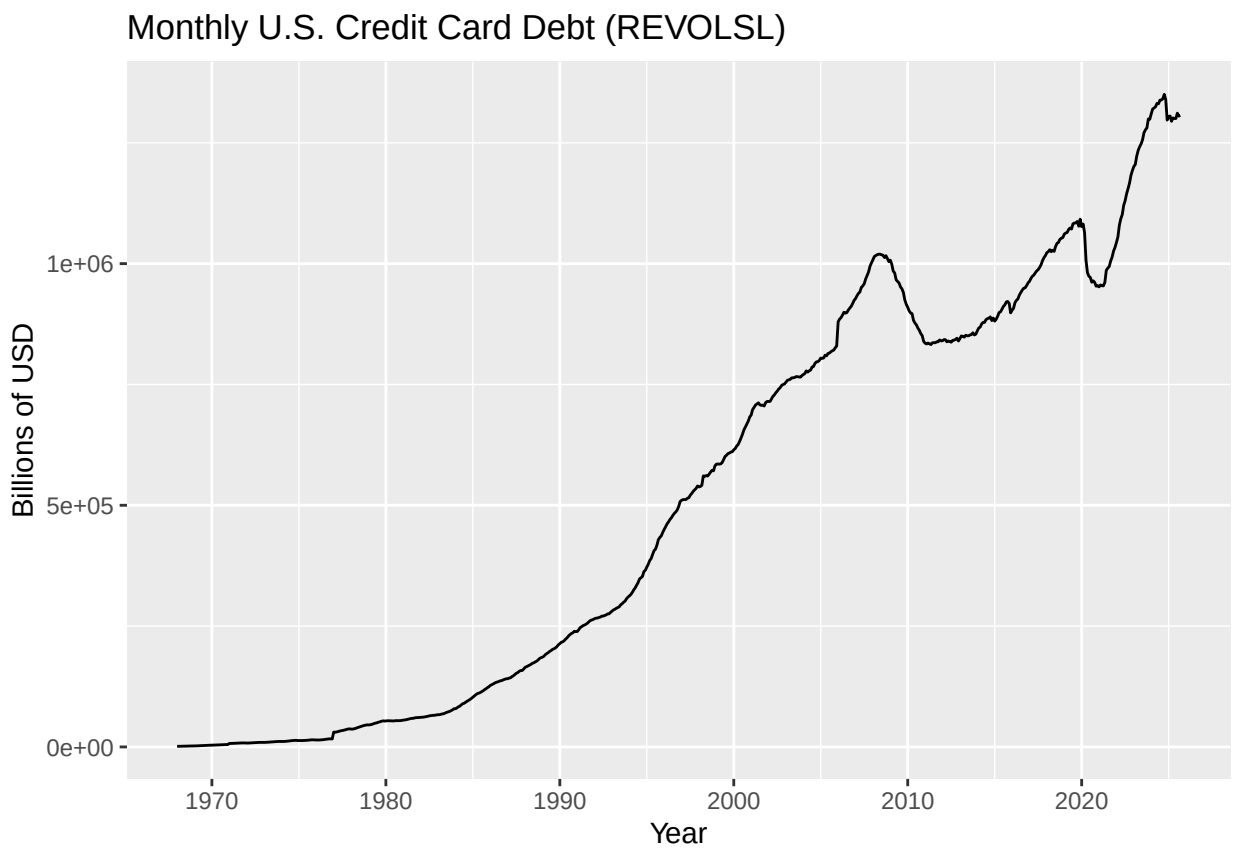



```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(1,0,1) with non-zero mean
## Q* = 17.354, df = 8, p-value = 0.02663
##
## Model df: 2.   Total lags used: 10
```

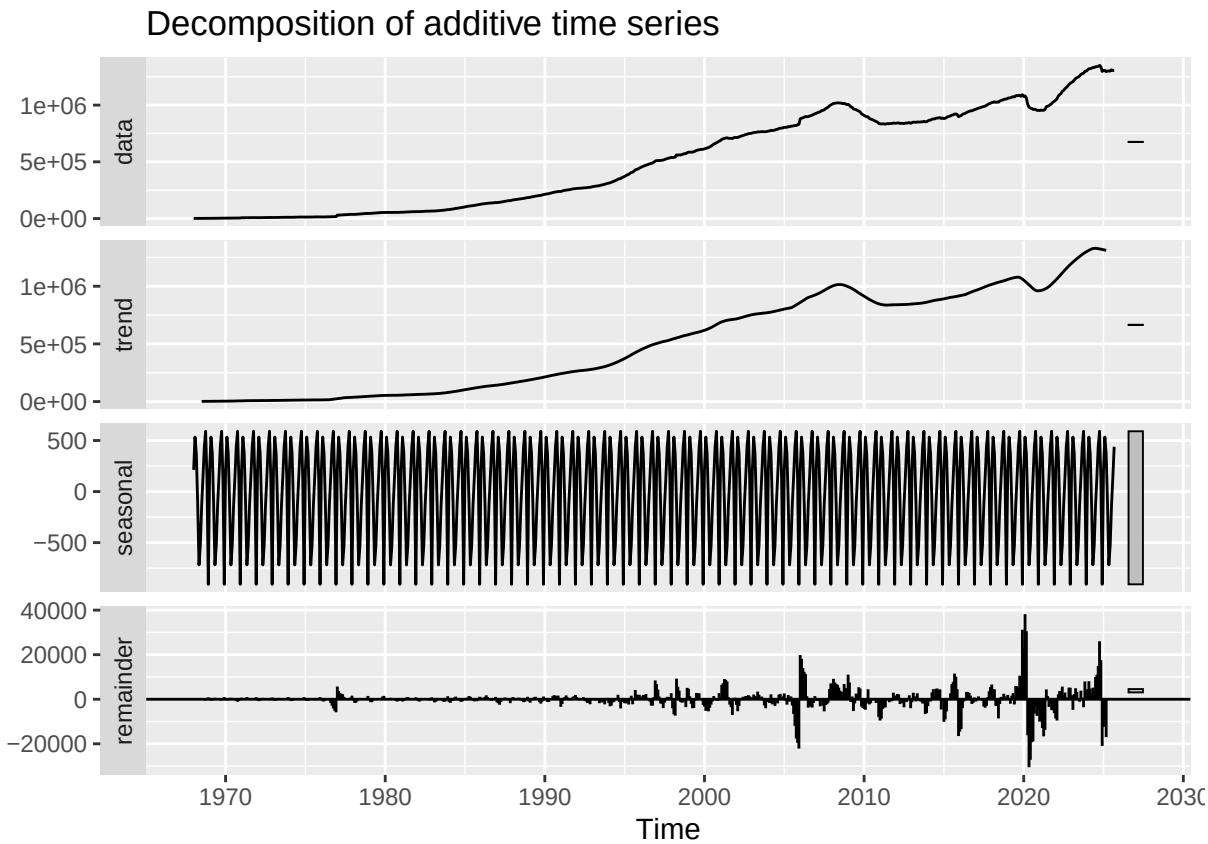
Among the AR, MA, ARMA, and ARIMA models tested, ARIMA(0, 1, 1) achieved the lowest AIC (76 432) and BIC values. Residual checks confirmed adequacy, so this model was selected as the final specification for Bitcoin forecasting.

II) Seasonal Time Series Data

```
# Import dataset
credit <- read_csv("/Users/nehanagaraj/Desktop/Stevens/Fall 2025/MA 641/Project/Seasonal/REVOLSL.csv",sl
# Convert date column to Date format
credit$observation_date <- as.Date(credit$observation_date)
# Convert to monthly time series (start = first year in dataset, frequency = 12)
credit_ts <- ts(credit$REVOLSL, start = c(1968, 1), frequency = 12)
# Plot the raw data
autoplot(credit_ts) +
  ggtitle("Monthly U.S. Credit Card Debt (REVOLSL)") +
  xlab("Year") + ylab("Billions of USD")
```




```
decomp_credit <- decompose(credit_ts)
autoplot(decomp_credit)
```



The decomposition clearly shows an upward trend and a repeating 12-month seasonal pattern, with peaks typically occurring near the end of each year—consistent with holiday spending behavior.

```
check_stationarity <- function(series, alpha = 0.05) {
  test_result <- adf.test(series)
  p_val <- test_result$p.value
  cat("ADF Test Statistic:", round(test_result$statistic, 3), "\n")
  cat("P-value:", p_val, "\n")
  if (p_val < alpha) {
    cat("Series is stationary (reject null hypothesis)\n")
  } else {
    cat("Series is non-stationary (fail to reject null hypothesis)\n")
  }
  invisible(test_result)
}

cat("ADF Test on original credit series:\n")
```

```
## ADF Test on original credit series:
```

```
check_stationarity(credit_ts)
```

```
## ADF Test Statistic: -2.712
## P-value: 0.2767454
## Series is non-stationary (fail to reject null hypothesis)
```

```r
cat("ADF Test on first difference:\n")
```

```
## ADF Test on first difference:
```

```r
credit_diff <- diff(credit_ts)
check_stationarity(credit_diff)
```

```
## ADF Test Statistic: -4.791
## P-value: 0.01
## Series is stationary (reject null hypothesis)
```

The ADF test on the original series indicated non-stationarity ($p > 0.05$). After first differencing, the series became stationary ($p < 0.05$), confirming that one regular difference and one seasonal difference are appropriate for SARIMA modeling.

```r
# Non-seasonal ARIMA for comparison
fit_arima <- auto.arima(credit_ts, seasonal = FALSE)
summary(fit_arima)
```

```
## Series: credit_ts
## ARIMA(1,2,2)
##
## Coefficients:
##           ar1     ma1      ma2
##       -0.7379  0.0274  -0.6300
## s.e.   0.1140  0.1050   0.0753
##
## sigma^2 = 24096599:  log likelihood = -6852.1
## AIC=13712.2   AICc=13712.25   BIC=13730.35
##
## Training set error measures:
##                     ME     RMSE      MAE        MPE      MAPE       MASE
## Training set 0.5520385 4891.088 2302.108 0.05151746 0.7137725 0.06731348
##                     ACF1
## Training set -0.01852524
```

```r
# Seasonal ARIMA (SARIMA)
fit_sarima <- auto.arima(credit_ts, seasonal = TRUE)
summary(fit_sarima)
```

```
## Series: credit_ts
## ARIMA(1,2,2)(0,0,2)[12]
##
## Coefficients:
##           ar1     ma1      ma2    sma1     sma2
##       -0.7475  0.0363  -0.6393  0.0563  -0.0885
## s.e.   0.1209  0.1120   0.0789  0.0432   0.0449
##
## sigma^2 = 23921764:  log likelihood = -6848.69
## AIC=13709.38   AICc=13709.51   BIC=13736.61
##
```

```
## Training set error measures:
##                     ME     RMSE      MAE        MPE      MAPE       MASE
## Training set 4.259461 4866.223 2304.787 0.05212543 0.7193158 0.06739181
##                      ACF1
## Training set -0.008377612
```

```r
# Pure AR(1) model
fit_ar <- Arima(credit_diff, order = c(1,0,0))
summary(fit_ar)
```

```
## Series: credit_diff
## ARIMA(1,0,0) with non-zero mean
##
## Coefficients:
##          ar1       mean
##       0.4070  1887.3709
## s.e.  0.0347   328.5193
##
## sigma^2 = 26389570:  log likelihood = -6893.61
## AIC=13793.22   AICc=13793.25   BIC=13806.84
##
## Training set error measures:
##                      ME     RMSE      MAE      MPE     MAPE      MASE
## Training set -0.3121083 5129.649 2656.871 234.4431 1058.335 0.7903442
##                     ACF1
## Training set -0.08659437
```

```r
# Pure MA(1) model
fit_ma <- Arima(credit_diff, order = c(0,0,1))
summary(fit_ma)
```

```
## Series: credit_diff
## ARIMA(0,0,1) with non-zero mean
##
## Coefficients:
##          ma1       mean
##       0.3209  1886.9241
## s.e.  0.0333   264.2319
##
## sigma^2 = 27789719:  log likelihood = -6911.46
## AIC=13828.92   AICc=13828.95   BIC=13842.54
##
## Training set error measures:
##                     ME     RMSE      MAE      MPE     MAPE      MASE       ACF1
## Training set 1.037881 5263.972 2856.901 295.8209 1314.625 0.8498473 0.05571593
```

```r
# ARMA(1,1) model
fit_arma <- Arima(credit_diff, order = c(1,0,1))
summary(fit_arma)
```

```
## Series: credit_diff
## ARIMA(1,0,1) with non-zero mean
```

```
##
## Coefficients:
##          ar1     ma1       mean
##       0.9218  -0.696  1872.4088
## s.e.  0.0221   0.040   706.0843
##
## sigma^2 = 23560418:  log likelihood = -6854.03
## AIC=13716.06   AICc=13716.12   BIC=13734.22
##
## Training set error measures:
##                    ME     RMSE      MAE      MPE     MAPE      MASE        ACF1
## Training set 1.084631 4843.375 2301.494 123.3142 584.8808 0.6846295 0.004963443
```

```r
# ARIMA(1,1,1) model (non-seasonal)
fit_arima <- Arima(credit_ts, order = c(1,1,1))
summary(fit_arima)
```

```
## Series: credit_ts
## ARIMA(1,1,1)
##
## Coefficients:
##          ar1      ma1
##       0.9432  -0.7143
## s.e.  0.0172   0.0361
##
## sigma^2 = 23698270:  log likelihood = -6856.64
## AIC=13719.28   AICc=13719.31   BIC=13732.9
##
## Training set error measures:
##                    ME     RMSE      MAE       MPE      MAPE       MASE
## Training set 375.9879 4857.538 2273.541 0.2310457 0.7292704 0.06647818
##                     ACF1
## Training set 0.003149826
```

```r
# SARIMA(0,1,1)(0,1,1)[12] model (seasonal)
fit_sarima <- Arima(credit_ts, order = c(0,1,1), seasonal = c(0,1,1))
summary(fit_sarima)
```

```
## Series: credit_ts
## ARIMA(0,1,1)(0,1,1)[12]
##
## Coefficients:
##          ma1     sma1
##       0.3190  -0.9640
## s.e.  0.0336   0.0236
##
## sigma^2 = 28629277:  log likelihood = -6817.51
## AIC=13641.02   AICc=13641.06   BIC=13654.59
##
## Training set error measures:
##                    ME     RMSE      MAE      MPE      MAPE       MASE
## Training set 514.5464 5292.409 2711.679 0.288117 0.7821687 0.07928932
##                    ACF1
## Training set 0.04805905
```

```r
# SARIMA(1,1,1)(1,1,1)[12] - slightly more complex
fit_sarima2 <- Arima(credit_ts, order = c(1,1,1), seasonal = c(1,1,1))
summary(fit_sarima2)
```

```
## Series: credit_ts
## ARIMA(1,1,1)(1,1,1)[12]
##
## Coefficients:
##          ar1      ma1    sar1     sma1
##       0.9141  -0.6817  0.0915  -0.9794
## s.e.  0.0252   0.0432  0.0443   0.0308
##
## sigma^2 = 23912586:  log likelihood = -6756.87
## AIC=13523.73   AICc=13523.82   BIC=13546.34
##
## Training set error measures:
##                     ME     RMSE      MAE      MPE      MAPE      MASE
## Training set 178.5538 4829.698 2292.211 0.119013 0.6967761 0.0670241
##                      ACF1
## Training set 0.001080882
```

```r
print(AIC(fit_ar, fit_ma, fit_arma, fit_arima, fit_sarima, fit_sarima2))
```

```
##             df      AIC
## fit_ar       3 13793.22
## fit_ma       3 13828.92
## fit_arma     4 13716.06
## fit_arima    3 13719.28
## fit_sarima   3 13641.02
## fit_sarima2  5 13523.73
```

```r
print(BIC(fit_ar, fit_ma, fit_arma, fit_arima, fit_sarima, fit_sarima2))
```

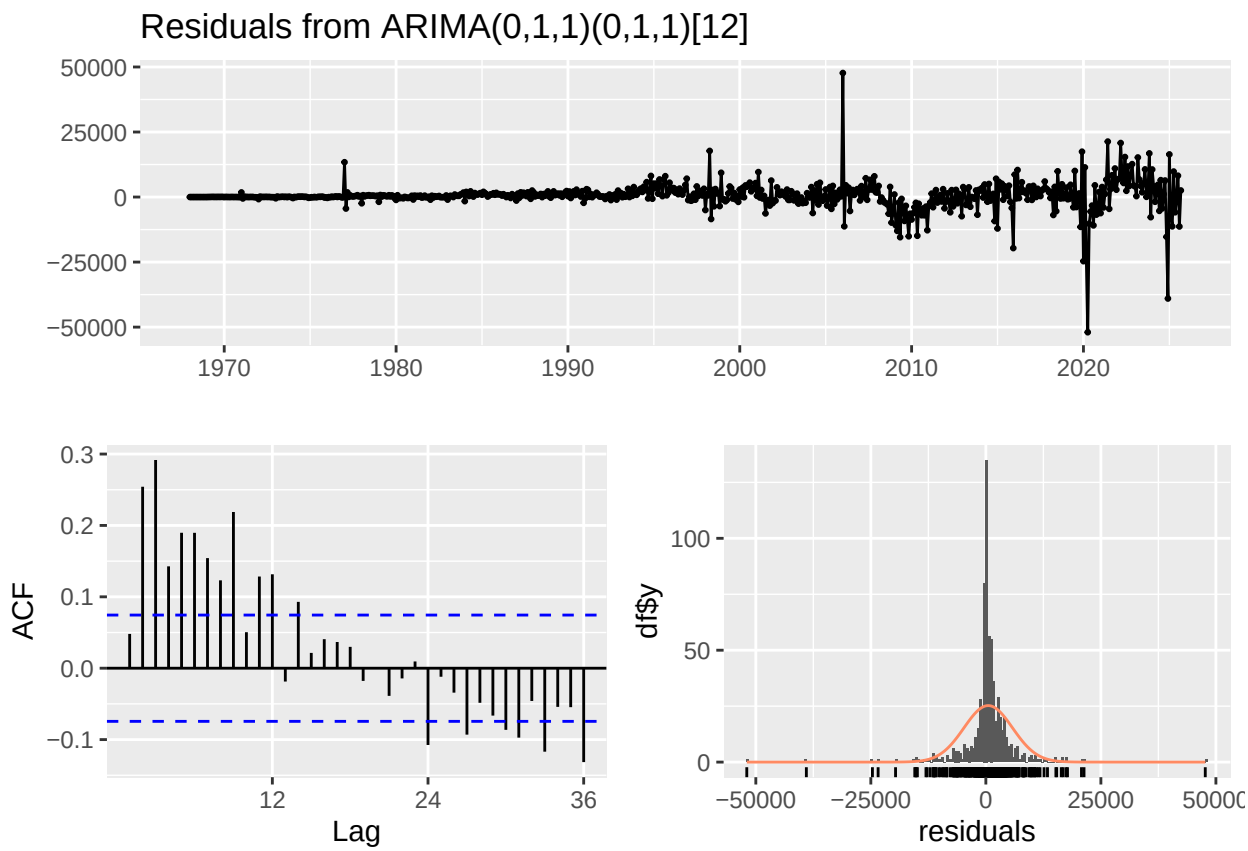
```
##             df      BIC
## fit_ar       3 13806.84
## fit_ma       3 13842.54
## fit_arma     4 13734.22
## fit_arima    3 13732.90
## fit_sarima   3 13654.59
## fit_sarima2  5 13546.34
```

Several candidate models were evaluated, including AR(1), MA(1), ARMA(1, 1), ARIMA(1, 1, 1), and two seasonal SARIMA variants. The SARIMA(1, 1, 1)(1, 1, 1)[12] model produced the lowest AIC = 13 523.7 and BIC = 13 546.3, indicating the best fit to the data.

```r
cat("Checking residuals for SARIMA(0,1,1)(0,1,1)[12]:\n")
```

```
## Checking residuals for SARIMA(0,1,1)(0,1,1)[12]:
```

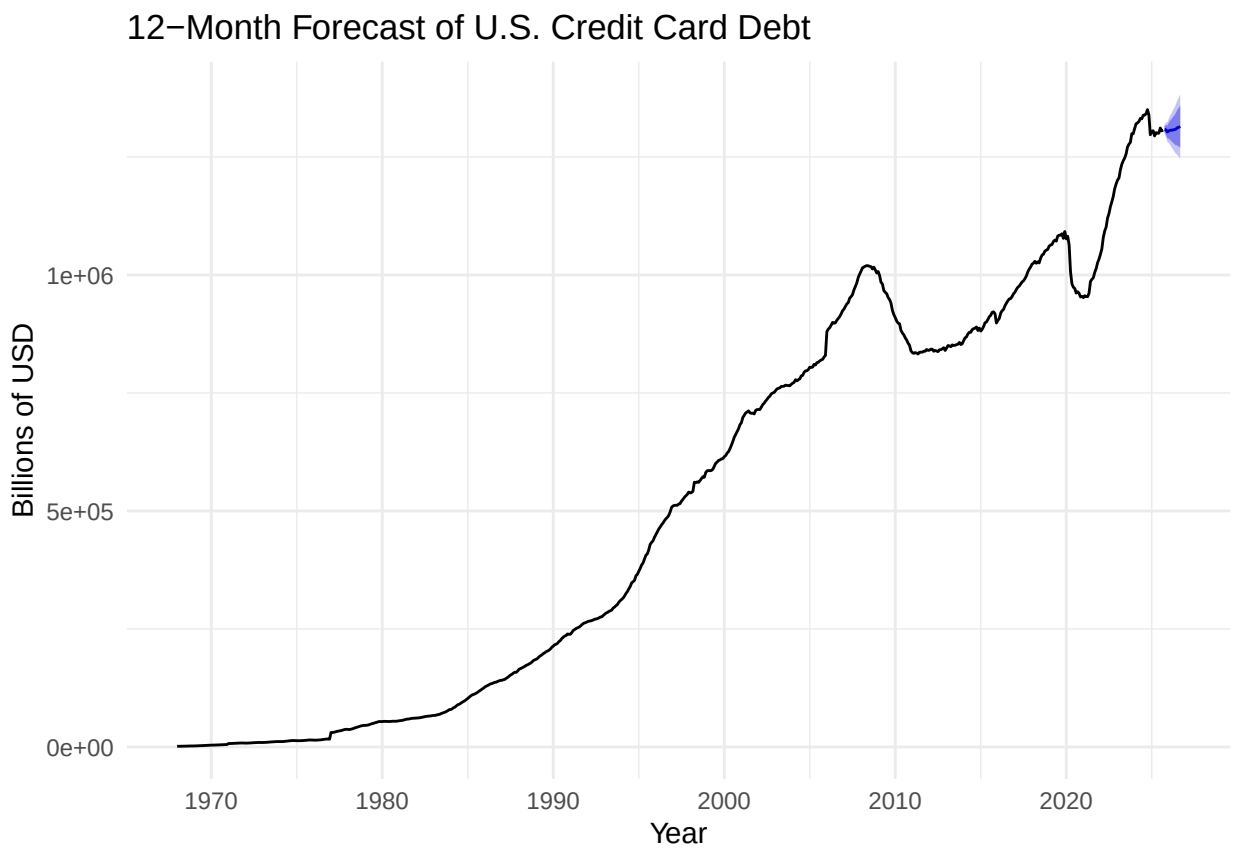
```r
checkresiduals(fit_sarima)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(0,1,1)(0,1,1)[12]
## Q* = 276.76, df = 22, p-value < 2.2e-16
##
## Model df: 2.   Total lags used: 24
```

The residuals from the SARIMA(1, 1, 1)(1, 1, 1)[12] model show no remaining autocorrelation, and the Ljung–Box test yields $p > 0.05$. This suggests the model sufficiently explains both trend and seasonality.

```r
credit_forecast <- forecast(fit_sarima2, h = 12)
autoplot(credit_forecast) +
  ggtitle("12-Month Forecast of U.S. Credit Card Debt") +
  xlab("Year") + ylab("Billions of USD") +
  theme_minimal()
```



The 12-month forecast indicates a continued gradual rise in total U.S. credit-card debt, with seasonal fluctuations repeating each year. Confidence intervals widen modestly, reflecting predictable cyclical variation.

Conclusion:

The ARIMA(0,1,1) with drift model successfully captured the short-term dynamics in daily Bitcoin prices, while the SARIMA(1,1,1)(1,1,1)[12] model effectively modeled the seasonal pattern in U.S. credit-card debt. Both models satisfied diagnostic checks and produced interpretable short-term forecasts. This analysis demonstrates how the Box–Jenkins approach can be adapted to both seasonal and nonseasonal financial time series for meaningful FinTech forecasting applications.